



TILERA[®]

MULTICORE DEVELOPMENT ENVIRONMENT

VECTOR LIBRARY API REFERENCE MANUAL

RELEASE 4.3.4.193953
DOC. No. UG217
OCTOBER 26, 2015
TILERA CORPORATION

© Copyright 2014 Tiler Corporation. All rights reserved. Printed in the United States of America.

The information contained in this document is the property of Tiler Corporation. It has been released under Non-Disclosure Agreement. Any unauthorized review, use, disclosure or distribution is strictly prohibited.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, except as may be expressly permitted by the applicable copyright statutes or in writing by the Publisher.

The following are registered trademarks of Tiler Corporation: Tiler and the Tiler logo. The following are trademarks of Tiler Corporation: Embedding Multicore, The Multicore Company, Tile Processor, TILE Architecture, TILE64, TILEPro, TILEPro36, TILEPro64, TILEExpress, TILEExpress-64, TILEExpressPro-64, TILEExpress-20G, TILEExpressPro-20G, TILEExpressPro-22G, iMesh, TileDirect, TILEmpower, TILEmpower-Gx, TILEncore, TILEncorePro, TILEncore-Gx, TILE-Gx, TILE-Gx16, TILE-Gx36, TILE-Gx64, TILE-Gx100, TILE-Gx3000, TILE-Gx5000, TILE-Gx8000, DDC (Dynamic Distributed Cache), Multicore Development Environment, Gentle Slope Programming, iLib, TMC (Tiler Multicore Components), hardwall, Zero Overhead Linux (ZOL), MiCA (Multicore iMesh Coprocessing Accelerator), and mPIPE (multicore Programmable Intelligent Packet Engine). All other trademarks and/or registered trademarks are the property of their respective owners.

Third-party software: The Tiler IDE makes use of the BeanShell scripting library. Source code for the BeanShell library can be found at the BeanShell website (<http://www.beanshell.org/developer.html>).

The following is a trademark of Marvell Semiconductor, Inc.: Distributed Switching Architecture (DSA).

This document contains advance information on Tiler products that are in development, sampling or initial production phases. This information and specifications contained herein are subject to change without notice at the discretion of Tiler Corporation.

No license, express or implied by estoppels or otherwise, to any intellectual property is granted by this document. Tiler disclaims any express or implied warranty relating to the sale and/or use of Tiler products, including liability or warranties relating to fitness for a particular purpose, merchantability or infringement of any patent, copyright or other intellectual property right.

Products described in this document are NOT intended for use in medical, life support, or other hazardous uses where malfunction could result in death or bodily injury.

THE INFORMATION CONTAINED IN THIS DOCUMENT IS PROVIDED ON AN “AS IS” BASIS. Tiler assumes no liability for damages arising directly or indirectly from any use of the information contained in this document.

Mon Oct 26 22:21:39 EDT 2015

Tiler Corporation
Information: info@tilera.com
Website: <http://www.tilera.com>

Contents

1	TILE Vector Library API Reference	1
2	Vector Library Examples	3
2.1	Examples	4
3	Module Documentation	5
3.1	Vector Data Types	5
3.1.1	Detailed Description	5
3.1.2	Typedef Documentation	6
3.2	Vector Instructions	7
3.2.1	Detailed Description	7
3.2.2	Function Documentation	9
3.3	Unaligned Data Access	28
3.3.1	Detailed Description	28
3.3.2	Define Documentation	28
4	Data Structure Documentation	31
4.1	_vec_UnalignedStruct Struct Reference	31
4.1.1	Detailed Description	31
4.1.2	Field Documentation	31
4.2	vec_Byte8Union Union Reference	32
4.2.1	Detailed Description	32
4.2.2	Field Documentation	32
4.3	vec_Half4Union Union Reference	33
4.3.1	Detailed Description	33

CONTENTS

4.3.2	Field Documentation	33
4.4	vec_Int2Union Union Reference	34
4.4.1	Detailed Description	34
4.4.2	Field Documentation	34
4.5	vec_UnalignedArray Struct Reference	35
4.5.1	Detailed Description	35
4.5.2	Field Documentation	35
4.6	vec_UnalignedPointer Struct Reference	36
4.6.1	Detailed Description	36
4.6.2	Field Documentation	36
5	Example Documentation	37
5.1	Unaligned_Data_Access	37
5.2	Vector_Add	39

Chapter 1

TILE Vector Library API Reference

This document describes the single-instruction, multiple-data (SIMD) intrinsics supported by the TILE(tm) architecture. The SIMD instructions operate on vectors of data, performing multiple operations with a single instruction. The TILE architecture supports a variety of operations on packed byte and halfword datatypes.

We recommend that new users begin with the [Vector Data Types](#) section and then proceed to the [Vector Instructions](#) section.

In addition to SIMD intrinsic operations, the Vector library also provides data structures and utility routines for convenient manipulation of packed datatypes; see the [Unaligned Data Access](#) section.

To use the Vector Library API, specify the following include file:

```
#include <vec_ops.h>
```


Chapter 2

Vector Library Examples

2.1 Examples

Here is a list of all examples:

- [Unaligned_Data_Access](#)
- [Vector_Add](#)

Chapter 3

Module Documentation

3.1 Vector Data Types

3.1.1 Detailed Description

Types and structures for accessing packed datatypes.

Data Structures

- union [vec_Byte8Union](#)

A union structure for convenient access to members of a vec_byte8.

- union [vec_Half4Union](#)

A union structure for convenient access to members of a vec_half4.

- union [vec_Int2Union](#)

A union structure for convenient access to members of a vec_int2.

Typedefs

- typedef uint64_t [vec_byte8](#)
- typedef uint64_t [vec_half4](#)
- typedef uint64_t [vec_int2](#)

3.1.2 Typedef Documentation

3.1.2.1 typedef uint64_t vec_byte8

A vector of eight bytes.

vec_byte8 type can represent either signed or unsigned byte data.

3.1.2.2 typedef uint64_t vec_half4

A vector of four halfwords.

vec_half4 type can represent either signed or unsigned halfword data.

3.1.2.3 typedef uint64_t vec_int2

A vector of two ints.

vec_int2 type can represent either signed or unsigned int data.

3.2 Vector Instructions

3.2.1 Detailed Description

Intrinsics for accessing the hardware's vector operations.

Functions

- `vec_byte4` [vec_vladd](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vladdi](#) (`vec_byte4 SrcA`, `int8_t Imm8`)
- `vec_byte4` [vec_vladduc](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `uint32_t` [vec_vladiffu](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlavg](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlcmpeq](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlcmpeq_i](#) (`vec_byte4 SrcA`, `int8_t Imm8`)
- `vec_byte4` [vec_vlcmples](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlcmples_u](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlcmplt](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlcmplt_i](#) (`vec_byte4 SrcA`, `int8_t Imm8`)
- `vec_byte4` [vec_vlcmplt_u](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlcmpltui](#) (`vec_byte4 SrcA`, `int8_t Imm8`)
- `vec_byte4` [vec_vlcmpne](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_half2` [vec_vlddotpu](#) (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t` [vec_vlddotpua](#) (`uint32_t Dest`, `uint32_t SrcA`, `uint32_t SrcB`)
- `vec_half2` [vec_vlddotpus](#) (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t` [vec_vlddotpusa](#) (`uint32_t Dest`, `uint32_t SrcA`, `uint32_t SrcB`)
- `vec_half2` [vec_vldotp](#) (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t` [vec_vldotpa](#) (`uint32_t Dest`, `uint32_t SrcA`, `uint32_t SrcB`)
- `vec_half2` [vec_vldotpu](#) (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t` [vec_vldotpua](#) (`uint32_t Dest`, `uint32_t SrcA`, `uint32_t SrcB`)
- `vec_half2` [vec_vldotpus](#) (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t` [vec_vldotpusa](#) (`uint32_t Dest`, `uint32_t SrcA`, `uint32_t SrcB`)
- `vec_byte4` [vec_vlint_h](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlint_l](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlmaxu](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlmaxui](#) (`vec_byte4 SrcA`, `int8_t Imm8`)
- `vec_byte4` [vec_vlminu](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlminui](#) (`vec_byte4 SrcA`, `int8_t Imm8`)
- `vec_byte4` [vec_vlmnz](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlmultu](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlmulu](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4` [vec_vlmulus](#) (`vec_byte4 SrcA`, `vec_byte4 SrcB`)

- `vec_byte4 vec_v1mz` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `uint32_t vec_v1sadau` (`uint32_t Dest`, `vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `uint32_t vec_v1sadu` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v1shl` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v1shli` (`vec_byte4 SrcA`, `unsigned int ShAmt`)
- `vec_byte4 vec_v1shrs` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v1shrsi` (`vec_byte4 SrcA`, `unsigned int ShAmt`)
- `vec_byte4 vec_v1shru` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v1shrui` (`vec_byte4 SrcA`, `unsigned int ShAmt`)
- `vec_byte4 vec_v1sub` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v1subuc` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_half2 vec_v2add` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2addi` (`vec_half2 SrcA`, `int8_t Imm8`)
- `vec_half2 vec_v2addsc` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t vec_v2adiffs` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2avgs` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2cmpeq` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2cmpeqi` (`vec_half2 SrcA`, `int8_t Imm8`)
- `vec_half2 vec_v2cmples` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2cmpleu` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2cmplts` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2cmpltsi` (`vec_half2 SrcA`, `int8_t Imm8`)
- `vec_half2 vec_v2cmpltu` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2cmpltui` (`vec_half2 SrcA`, `int8_t Imm8`)
- `vec_half2 vec_v2cmpne` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2dotp` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t vec_v2dotpa` (`uint32_t Dest`, `uint32_t SrcA`, `uint32_t SrcB`)
- `vec_half2 vec_v2int_h` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2int_l` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2maxs` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2maxsi` (`vec_half2 SrcA`, `int8_t Imm8`)
- `vec_half2 vec_v2mins` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2minsi` (`vec_half2 SrcA`, `int8_t Imm8`)
- `vec_half2 vec_v2mnz` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2mulpsc` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2muls` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2mults` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_half2 vec_v2mz` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `vec_byte4 vec_v2packh` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v2packl` (`vec_byte4 SrcA`, `vec_byte4 SrcB`)
- `vec_byte4 vec_v2packuc` (`vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t vec_v2sadas` (`uint32_t Dest`, `vec_half2 SrcA`, `vec_half2 SrcB`)
- `uint32_t vec_v2sadau` (`uint32_t Dest`, `vec_half2 SrcA`, `vec_half2 SrcB`)

- uint32_t `vec_v2sads` (vec_half2 SrcA, vec_half2 SrcB)
- uint32_t `vec_v2sadu` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v2shl` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v2shli` (vec_half2 SrcA, unsigned int ShAmt)
- vec_half2 `vec_v2shlsc` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v2shrs` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v2shrsi` (vec_half2 SrcA, unsigned int ShAmt)
- vec_half2 `vec_v2shru` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v2shrui` (vec_half2 SrcA, unsigned int ShAmt)
- vec_half2 `vec_v2sub` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v2subsc` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4add` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4addsc` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4int_h` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4int_l` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4packsc` (uint32_t SrcA, uint32_t SrcB)
- vec_half2 `vec_v4shl` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4shlsc` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4shrs` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4shru` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4sub` (vec_half2 SrcA, vec_half2 SrcB)
- vec_half2 `vec_v4subsc` (vec_half2 SrcA, vec_half2 SrcB)

3.2.2 Function Documentation

3.2.2.1 `vec_byte4 vec_v1add (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Add.

Add the eight bytes in the first source operand to the eight bytes in the second source operand. This operation has a 1 cycle latency.

3.2.2.2 `vec_byte4 vec_v1addi (vec_byte4 SrcA, int8_t Imm8)`

Vector One Byte Add Immediate.

Add an immediate to all four of the bytes in the source operand. This operation has a 1 cycle latency.

3.2.2.3 `vec_byte4 vec_v1adduc (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Add Unsigned Clamped.

Add the eight bytes in the first source operand to the eight bytes in the second source operand and clamp each result to zero or the maximum positive value. This operation has a 1 cycle latency.

3.2.2.4 `uint32_t vec_v1diffu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Absolute Difference Unsigned.

Compute the absolute differences between the eight bytes in the first source operand and the eight bytes in the second source operand. This operation has a 2 cycle latency.

3.2.2.5 `vec_byte4 vec_v1avgu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Average Unsigned.

Compute the average of the eight bytes in the first source operand and the eight bytes in the second source operand, rounding upwards. This operation has a 1 cycle latency.

3.2.2.6 `vec_byte4 vec_v1cmpeq (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Set Equal To.

Sets each result byte to one if the corresponding byte of the first source operand is equal to the byte of the second source operand. Otherwise the result is set to zero. This instruction treats both source bytes as signed values. This operation has a 1 cycle latency.

3.2.2.7 `vec_byte4 vec_v1cmpeqi (vec_byte4 SrcA, int8_t Imm8)`

Vector One Byte Set Equal To Immediate.

Sets each result byte to one if the corresponding byte of the first source operand is equal to a sign extended immediate. Otherwise the result is set to zero. This instruction treats both source bytes as signed values. This operation has a 1 cycle latency.

3.2.2.8 `vec_byte4 vec_v1cmple (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Set Less Than or Equal.

Sets each result byte to one if the corresponding byte of the first source operand is less than or equal to the byte of the second source operand. Otherwise the result is set to zero. This instruction treats both source bytes as signed values. This operation has a 1 cycle latency.

3.2.2.9 `vec_byte4 vec_v1cmpleu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Set Less Than or Equal Unsigned.

Sets each result byte to one if the corresponding byte of the first source operand is less than or equal to the byte of the second source operand. Otherwise the result is set to zero. This instruction treats both source bytes as unsigned values. This operation has a 1 cycle latency.

3.2.2.10 vec_byte4 vec_v1cmplts (vec_byte4 SrcA, vec_byte4 SrcB)

Vector One Byte Set Less Than.

Sets each result byte to one if the corresponding byte of the first source operand is less than the byte of the second source operand. Otherwise the result is set to zero. This instruction treats both source bytes as signed values. This operation has a 1 cycle latency.

3.2.2.11 vec_byte4 vec_v1cmpltsi (vec_byte4 SrcA, int8_t Imm8)

Vector One Byte Set Less Than Immediate.

Sets each result byte to one if the corresponding byte of the first source operand is less than a sign extended immediate. Otherwise the result is set to zero. This instruction treats both source bytes as signed values. This operation has a 1 cycle latency.

3.2.2.12 vec_byte4 vec_v1cmpltu (vec_byte4 SrcA, vec_byte4 SrcB)

Vector One Byte Set Less Than Unsigned.

Sets each result byte to one if the corresponding byte of the first source operand is less than the byte of the second source operand. Otherwise the result is set to zero. This instruction treats both source bytes as unsigned values. This operation has a 1 cycle latency.

3.2.2.13 vec_byte4 vec_v1cmpltui (vec_byte4 SrcA, int8_t Imm8)

Vector One Byte Set Less Than Unsigned Immediate.

Sets each result byte to one if the corresponding byte of the first source operand is less than a sign extended immediate. Otherwise the result is set to zero. This instruction treats both source bytes as unsigned values. This operation has a 1 cycle latency.

3.2.2.14 vec_byte4 vec_v1cmpne (vec_byte4 SrcA, vec_byte4 SrcB)

Vector One Byte Set Not Equal To.

Sets each result byte to one if the corresponding byte of the first source operand is not equal to the byte of the second source operand. Otherwise the result is set to zero. This instruction treats both source bytes as signed values. This operation has a 1 cycle latency.

3.2.2.15 vec_half2 vec_v1ddotpu (vec_half2 SrcA, vec_half2 SrcB)

Vector One Byte Dual Dot Product Unsigned.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand. The low 32-bits of the result is set to the sum of products of the low-order four 8-bit quantities, and the high 32-bits of the result is set to the sum of the products of the high-order four 8-bit quantities. The quantities in the operands are treated as unsigned values. This operation has a 2 cycle latency.

3.2.2.16 `uint32_t vec_v1ddotpua (uint32_t Dest, uint32_t SrcA, uint32_t SrcB)`

Vector One Byte Dual Dot Product Unsigned and Accumulate.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand. The low 32-bits of the result is set to the sum low-order 32-bit quantity of the desination and the products of the low-order four 8-bit quantities, and the high 32-bits of the result is set to the sum of the high-order 32-bit quantity of the destination and the products of the high-order four 8-bit quantities. The quantities in the operands are treated as unsigned values. This operation has a 2 cycle latency.

3.2.2.17 `vec_half2 vec_v1ddotpus (vec_half2 SrcA, vec_half2 SrcB)`

Vector One Byte Dual Dot Product Unsigned Signed.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand. The low 32-bits of the result is set to the sum of products of the low-order four 8-bit quantities, and the high 32-bits of the result is set to the sum of the products of the high-order four 8-bit quantities. The quantities in the first operand are treated as unsigned values, and the quantities in the second operand are treated as signed values. This operation has a 2 cycle latency.

3.2.2.18 `uint32_t vec_v1ddotpusa (uint32_t Dest, uint32_t SrcA, uint32_t SrcB)`

Vector One Byte Dual Dot Product Unsigned Signed and Accumulate.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand. The low 32-bits of the result is set to the sum low-order 32-bit quantity of the desination and the products of the low-order four 8-bit quantities, and the high 32-bits of the result is set to the sum of the high-order 32-bit quantity of the destination and the products of the high-order four 8-bit quantities. The quantities in the first operand are treated as unsigned values, and the quantities in the second operand are treated as signed values. This operation has a 2 cycle latency.

3.2.2.19 `vec_half2 vec_v1dotp (vec_half2 SrcA, vec_half2 SrcB)`

Vector One Byte Dot Product.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand and sum the products. This operation has a 2 cycle latency.

3.2.2.20 uint32_t vec_v1dotpa (uint32_t Dest, uint32_t SrcA, uint32_t SrcB)

Vector One Byte Dot Product and Accumulate.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand and sum the products and the destination operand. This operation has a 2 cycle latency.

3.2.2.21 vec_half2 vec_v1dotpu (vec_half2 SrcA, vec_half2 SrcB)

Vector One Byte Dot Product Unsigned.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand and sum the products. The quantities in the operands are treated as unsigned values. This operation has a 2 cycle latency.

3.2.2.22 uint32_t vec_v1dotpua (uint32_t Dest, uint32_t SrcA, uint32_t SrcB)

Vector One Byte Dot Product Unsigned and Accumulate.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand and sum the products and the destination operand. The quantities in the operands are treated as unsigned values. This operation has a 2 cycle latency.

3.2.2.23 vec_half2 vec_v1dotpus (vec_half2 SrcA, vec_half2 SrcB)

Vector One Byte Dot Product Unsigned Signed.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand and sum the products. The quantities in the first operand are treated as unsigned values, and the quantities in the second operand are treated as signed values. This operation has a 2 cycle latency.

3.2.2.24 uint32_t vec_v1dotpusa (uint32_t Dest, uint32_t SrcA, uint32_t SrcB)

Vector One Byte Dot Product Unsigned Signed and Accumulate.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand and sum the products and the destination operand. The quantities in the first operand are treated as unsigned values, and the quantities in the second operand are treated as signed values. This operation has a 2 cycle latency.

3.2.2.25 vec_byte4 vec_v1int_h (vec_byte4 SrcA, vec_byte4 SrcB)

Vector One Byte Interleave High.

Interleave the four high-order bytes of the first operand with the four high-order bytes of the second operand. The high-order byte of the result will be the high-order byte of the first operand. For example if the first operand contains the packed bytes {A7,A6,A5,A4,A3,A2,A1,A0} and the second operand contains the packed bytes {B7,B6,B5,B4,B3,B2,B1,B0} then the result will be {A7,B7,A6,B6,A5,B5,A4,B4}. This operation has a 1 cycle latency.

3.2.2.26 `vec_byte4 vec_v1int_l (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Interleave Low.

Interleave the four low-order bytes of the first operand with the four low-order bytes of the second operand. The low-order byte of the result will be the low-order byte of the second operand. For example if the first operand contains the packed bytes {A7,A6,A5,A4,A3,A2,A1,A0} and the second operand contains the packed bytes {B7,B6,B5,B4,B3,B2,B1,B0} then the result will be {A3,B3,A2,B2,A1,B1,A0,B0}. This operation has a 1 cycle latency.

3.2.2.27 `vec_byte4 vec_v1maxu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Maximum Unsigned.

Set each byte in the destination to the maximum of the corresponding byte in the first source operand and the corresponding byte in the second source operand. This operation has a 1 cycle latency.

3.2.2.28 `vec_byte4 vec_v1maxui (vec_byte4 SrcA, int8_t Imm8)`

Vector One Byte Maximum Unsigned Immediate.

Set each byte in the destination to the maximum of the corresponding byte in the first source operand and the sign extended immediate. This operation has a 1 cycle latency.

3.2.2.29 `vec_byte4 vec_v1minu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Minimum Unsigned.

Set each byte in the destination to the minimum of the corresponding byte in the first source operand and the corresponding byte in the second source operand. This operation has a 1 cycle latency.

3.2.2.30 `vec_byte4 vec_v1minui (vec_byte4 SrcA, int8_t Imm8)`

Vector One Byte Minimum Unsigned Immediate.

Set each bytes in the destination to the minimum of the corresponding byte in the first source operand and the sign extended immediate. This operation has a 1 cycle latency.

3.2.2.31 `vec_byte4 vec_v1mnz (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Mask Not Zero.

Set each byte in the destination to the corresponding byte of the second operand if the corresponding byte of the first operand is not zero, otherwise set it to zero (0). This operation has a 1 cycle latency.

3.2.2.32 `vec_byte4 vec_v1multu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Multiply and Truncate Unsigned.

Multiply the eight 8-bit quantities in the first source operand by the eight 8-bit quantities in the second source operand. The result is truncated to the low-order 8 bits of the 16-bit product. This operation has a 2 cycle latency.

3.2.2.33 `vec_byte4 vec_v1mulu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Multiply Unsigned.

Multiply the four low-order 8-bit quantities in the first source operand by the four low-order 8-bit quantities in the second source operand. The 16-bit results are packed. This operation has a 2 cycle latency.

3.2.2.34 `vec_byte4 vec_v1mulus (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Multiply Unsigned Signed.

Multiply the four low-order 8-bit quantities in the first source operand by the four low-order 8-bit quantities in the second source operand. The 16-bit results are packed. The quantities in the first operand are treated as unsigned values, and the quantities in the second operand are treated as signed values. This operation has a 2 cycle latency.

3.2.2.35 `vec_byte4 vec_v1mz (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Mask Zero.

Set each byte in the destination to the corresponding byte of the second operand if the corresponding byte of the first operand is zero, otherwise set it to zero (0). This operation has a 1 cycle latency.

3.2.2.36 `uint32_t vec_v1sadau (uint32_t Dest, vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Sum of Absolute Difference Accumulate Unsigned.

Sum the absolute differences between the eight bytes in the first source operand and the eight bytes in the second source operand and accumulate the sum into the destination register. This operation has a 2 cycle latency.

3.2.2.37 `uint32_t vec_v1sadu (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Sum of Absolute Difference Unsigned.

Sum the absolute differences between the eight bytes in the first source operand and the eight bytes in the second source operand. This operation has a 2 cycle latency.

3.2.2.38 `vec_byte4 vec_v1shl (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Shift Left.

Logically shift the eight bytes in the first source operand to the left by the second source operand. The effective shift amount is the specified operand modulo the number of bits in a byte. Logical left shift shifts zeros into the low ordered bits in a byte. This operation has a 1 cycle latency.

3.2.2.39 `vec_byte4 vec_v1shli (vec_byte4 SrcA, unsigned int ShAmt)`

Vector One Byte Shift Left Immediate.

Logically shift the eight bytes in the first source operand to the left by an immediate. The effective shift amount is the specified immediate modulo the number of bits in a byte. Left shifts shift zeros into the low ordered bits in a byte and are suitable to be used as unsigned multiplication by powers of two. This operation has a 1 cycle latency.

3.2.2.40 `vec_byte4 vec_v1shrs (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Shift Right Signed.

Arithmetically shift the eight bytes in the first source operand to the right by the second source operand. The effective shift amount is the specified operand modulo the number of bits in a byte. Arithmetic right shift shifts the high ordered bit into the high ordered bits in a byte. This operation has a 1 cycle latency.

3.2.2.41 `vec_byte4 vec_v1shrsi (vec_byte4 SrcA, unsigned int ShAmt)`

Vector One Byte Shift Right Signed Immediate.

Arithmetically shift the eight bytes in the first source operand to the right by an immediate. The effective shift amount is the specified immediate modulo the number of bits in a byte. Arithmetic right shifts shift the high ordered bit into the high ordered bits in a byte. This operation has a 1 cycle latency.

3.2.2.42 `vec_byte4 vec_v1shru (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Shift Right Unsigned.

Logically shift the eight bytes in the first source operand to the right by the second source operand. The effective shift amount is the specified operand modulo the number of bits in a byte. Logical right shift shifts zeros into the high ordered bits in a byte. This operation has a 1 cycle latency.

3.2.2.43 `vec_byte4 vec_v1shrui (vec_byte4 SrcA, unsigned int ShAmt)`

Vector One Byte Shift Right Unsigned Immediate.

Logically shift the eight bytes in the first source operand to the right by an immediate. The effective shift amount is the specified immediate modulo the number of bits in a byte. Logical right shifts shift zeros into the high ordered bits in a byte and are suitable to be used as unsigned integer division by powers of two. This operation has a 1 cycle latency.

3.2.2.44 `vec_byte4 vec_v1sub (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Subtract.

Subtract the eight bytes in the second source operand from the eight bytes in the first source operand. This operation has a 1 cycle latency.

3.2.2.45 `vec_byte4 vec_v1subuc (vec_byte4 SrcA, vec_byte4 SrcB)`

Vector One Byte Subtract Unsigned Clamped.

Subtract the eight bytes in the second source operand from the eight bytes in the first source operand and clamp each result to zero or the maximum positive value. This operation has a 1 cycle latency.

3.2.2.46 `vec_half2 vec_v2add (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Add.

Add the four 16-bit quantities in the first source operand to the four 16-bit quantities in the second source operand. This operation has a 1 cycle latency.

3.2.2.47 `vec_half2 vec_v2addi (vec_half2 SrcA, int8_t Imm8)`

Vector Two Byte Add Immediate.

Add a sign extended immediate to both of the 16-bit quantities in the source operand. This operation has a 1 cycle latency.

3.2.2.48 `vec_half2 vec_v2addsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Add Signed Clamped.

Add the four 16-bit quantities in the first source operand to the four 16-bit quantities in the second source operand and clamp each result to the minimum negative or maximum positive 16-bit value. This operation has a 1 cycle latency.

3.2.2.49 `uint32_t vec_v2adiffs (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Absolute Difference Signed.

Compute the absolute differences between the four 16-bit quantities in the first source operand and the four 16-bit quantities in the second source operand. This operation has a 2 cycle latency.

3.2.2.50 `vec_half2 vec_v2avgs (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Average Signed.

Compute the average between the four 16-bit quantities in the first source operand and the four 16-bit quantities in the second source operand, rounding upwards. This operation has a 1 cycle latency.

3.2.2.51 `vec_half2 vec_v2cmpeq (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Set Equal To.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is equal to the corresponding 16-bit quantity of the second source operand. Otherwise the result is set to zero. This operation has a 1 cycle latency.

3.2.2.52 `vec_half2 vec_v2cmpeqi (vec_half2 SrcA, int8_t Imm8)`

Vector Two Byte Set Equal To Immediate.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is equal to a sign extended immediate. Otherwise the result is set to zero. This operation has a 1 cycle latency.

3.2.2.53 `vec_half2 vec_v2cmples (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Set Less Than or Equal.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is less than or equal to the corresponding 16-bit quantity of the second source operand. Otherwise the result is set to zero. This instruction treats both source 16-bit quantities as signed values. This operation has a 1 cycle latency.

3.2.2.54 `vec_half2 vec_v2cmpleu (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Set Less Than or Equal Unsigned.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is less than or equal to the corresponding 16-bit quantity of the second source operand. Otherwise the result is set to zero. This instruction treats both source 16-bit quantity as unsigned values. This operation has a 1 cycle latency.

3.2.2.55 `vec_half2 vec_v2cmplts (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Set Less Than.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is less than the corresponding 16-bit quantity of the second source operand. Otherwise the result is set to zero. This instruction treats both source 16-bit quantities as signed values. This operation has a 1 cycle latency.

3.2.2.56 `vec_half2 vec_v2cmpltsi (vec_half2 SrcA, int8_t Imm8)`

Vector Two Byte Set Less Than Immediate.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is less than a sign extended immediate. Otherwise the result is set to zero. This instruction treats the first source operand as 16-bit quantities as signed values. This operation has a 1 cycle latency.

3.2.2.57 `vec_half2 vec_v2cmpltu (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Set Less Than Unsigned.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is less than the corresponding 16-bit quantity of the second source operand. Otherwise the result is set to zero. This instruction treats both source 16-bit quantities as unsigned values. This operation has a 1 cycle latency.

3.2.2.58 `vec_half2 vec_v2cmpltui (vec_half2 SrcA, int8_t Imm8)`

Vector Two Byte Set Less Than Unsigned Immediate.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is less than a sign extended immediate. Otherwise the result is set to zero. This instruction treats the first source operand 16-bit quantities as unsigned values. This operation has a 1 cycle latency.

3.2.2.59 `vec_half2 vec_v2cmpne (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Set Not Equal To.

Sets each result 16-bit quantity to one if the corresponding 16-bit quantity of the first source operand is not equal to the 16-bit quantity of the second source operand. Otherwise the result is set to zero. This operation has a 1 cycle latency.

3.2.2.60 `vec_half2 vec_v2dotp (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Dot Product.

Multiply the four 16-bit quantities in the first source operand by the four 16-bit quantities in the second source operand and sum the products. This operation has a 2 cycle latency.

3.2.2.61 `uint32_t vec_v2dotpa (uint32_t Dest, uint32_t SrcA, uint32_t SrcB)`

Vector Two Byte Dot Product and Accumulate.

Multiply the four 16-bit quantities in the first source operand by the four 16-bit quantities in the second source operand and sum the products and the destination operand. This operation has a 2 cycle latency.

3.2.2.62 `vec_half2 vec_v2int_h (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Interleave High.

Interleave the two high-order 16-bit quantities of the first operand with the two high-order 16-bit quantities of the second operand. The high-order 16-bits of the result will be the high-order 16-bits of the first operand. For example if the first operand contains the packed 16-bit quantities {A3,A2,A1,A0} and the second operand contains the packed 16-bit quantities {B3,B2,B1,B0} then the result will be {A3,B3,A2,B2}. This operation has a 1 cycle latency.

3.2.2.63 `vec_half2 vec_v2int_l (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Interleave Low.

Interleave the two low-order 16-bit quantities of the first operand with the two low-order 16-bit quantities of the second operand. The low-order 16-bits of the result will be the low-order 16-bits of the second operand. For example if the first operand contains the packed 16-bit quantities {A3,A2,A1,A0} and the second operand contains the packed 16-bit quantities {B3,B2,B1,B0} then the result will be {A1,B1,A0,B0}. This operation has a 1 cycle latency.

3.2.2.64 `vec_half2 vec_v2maxs (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Maximum Signed.

Set each 16-bit quantity in the destination to the maximum of the corresponding 16-bit quantity in the first source operand and the corresponding 16-bit quantity in the second source operand. This operation has a 1 cycle latency.

3.2.2.65 `vec_half2 vec_v2maxsi (vec_half2 SrcA, int8_t Imm8)`

Vector Two Byte Maximum Signed Immediate.

Set each 16-bit quantity in the destination to the maximum of the corresponding 16-bit quantity in the first source operand and the sign extended immediate. This operation has a 1 cycle latency.

3.2.2.66 `vec_half2 vec_v2mins (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Minimum Signed.

Set each 16-bit quantity in the destination to the minimum of the corresponding 16-bit quantity in the first source operand and the corresponding 16-bit quantity in the second source operand. This operation has a 1 cycle latency.

3.2.2.67 `vec_half2 vec_v2minsi (vec_half2 SrcA, int8_t Imm8)`

Vector Two Byte Minimum Signed Immediate.

Set each 16-bit quantity in the destination to the minimum of the corresponding 16-bit quantity in the first source operand and the sign extended immediate. This operation has a 1 cycle latency.

3.2.2.68 `vec_half2 vec_v2mnz (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Mask Not Zero.

Set each 16-bit quantity in the destination to the corresponding 16-bit quantity of the second operand if the corresponding 16-bit quantity of the first operand is not zero, otherwise set it to zero (0). This operation has a 1 cycle latency.

3.2.2.69 `vec_half2 vec_v2mulpsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Multiply Fixed point Signed Clamped.

Multiply the two low-order 16-bit quantities in the first source operand by the two low-order 16-bit quantities in the second source operand. The multiplier result is shifted left 1 bit and clamped to the maximum positive

32-bit value. The operands are treated as 16-bit signed fractions with the decimal point below the sign bit. This operation has a 2 cycle latency.

3.2.2.70 **vec_half2 vec_v2mults (vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Multiply Signed.

Multiply the two low-order 16-bit quantities in the first source operand by the two low-order 16-bit quantities in the second source operand. The two 32-bit multiplication results are packed into 64-bits. This operation has a 2 cycle latency.

3.2.2.71 **vec_half2 vec_v2mults (vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Multiply and Truncate Signed.

Multiply the four 16-bit quantities in the first source operand by the four 16-bit quantities in the second source operand. The multiplier result is truncated to the low-order 16-bits of the 32-bit product and packed. This operation has a 2 cycle latency.

3.2.2.72 **vec_half2 vec_v2mz (vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Mask Zero.

Set each 16-bit quantity in the destination to the corresponding 16-bit quantity of the second operand if the corresponding 16-bit quantity of the first operand is zero, otherwise set it to zero (0). This operation has a 1 cycle latency.

3.2.2.73 **vec_byte4 vec_v2packh (vec_byte4 SrcA, vec_byte4 SrcB)**

Vector Two Bytes Pack High Byte.

Pack the high-order byte of each of the packed 16-bit quantities of the two source registers into the destination register. The high-order byte of the destination will be the high-order byte of the first operand. For example if the first operand contains the packed bytes {A3_1,A3_0,A2_1,A2_0,A1_1,A1_0,A0_1,A0_0} and the second operand contains the packed bytes {B3_1,B3_0,B2_1,B2_0,B1_1,B1_0,B0_1,B0_0} then the result will be {A3_1,A2_1,A1_1,A0_1,B3_1,B2_1,B1_1,B0_1}. This operation has a 1 cycle latency.

3.2.2.74 **vec_byte4 vec_v2packl (vec_byte4 SrcA, vec_byte4 SrcB)**

Vector Two Byte Pack Low Byte.

Pack the low-order byte of each of the packed 16-bit quantities of the two source registers into the destination register. The low-order byte of the destination will be the low-order byte of the second operand. For example if the first operand contains the packed bytes {A3_1,A3_0,A2_1,A2_0,A1_1,A1_0,A0_1,A0_0}

and the second operand contains the packed bytes {B3_1,B3_0,B2_1,B2_0,B1_1,B1_0,B0_1,B0_0} then the result will be {A3_0,A2_0,A1_0,A0_0,B3_0,B2_0,B1_0,B0_0}. This operation has a 1 cycle latency.

3.2.2.75 **vec_byte4 vec_v2packuc (vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Pack Unsigned Clamped.

Clamp each 16-bit quantity of the two source registers to the maximum positive or zero byte value, and then pack the results into the destination register. The high-order byte of the destination will be the clamped high-order 16-bit quantity of the first operand and the low-order byte of the destination will be the clamped low-order 16-bit quantity of the second operand. For example if the first operand contains the packed 16-bit quantities {A3,A2,A1,A0} and the second operand contains the packed quantities {B3,B2,B1,B0} then the result will be {sat_A3,sat_A2,sat_A1,sat_A0,sat_B3,sat_B2,sat_B1,sat_B0}. This operation has a 1 cycle latency.

3.2.2.76 **uint32_t vec_v2sadas (uint32_t Dest, vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Sum of Absolute Difference Accumulate Signed.

Sum the absolute differences between the four 16-bit quantities in the first source operand and the four 16-bit quantities in the second source operand and accumulate the sum into the destination register. This operation has a 2 cycle latency.

3.2.2.77 **uint32_t vec_v2sadau (uint32_t Dest, vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Sum of Absolute Difference Accumulate Unsigned.

Sum the absolute differences between the four 16-bit quantities in the first source operand and the four 16-bit quantities in the second source operand and accumulate the sum into the destination register. This operation has a 2 cycle latency.

3.2.2.78 **uint32_t vec_v2sads (vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Sum of Absolute Difference Signed.

Sum the absolute differences between the four 16-bit quantities in the first source operand and the four 16-bit quantities in the second source operand. This operation has a 2 cycle latency.

3.2.2.79 **uint32_t vec_v2sadu (vec_half2 SrcA, vec_half2 SrcB)**

Vector Two Byte Sum of Absolute Difference Unsigned.

Sum the absolute differences between the four 16-bit quantities in the first source operand and the four 16-bit quantities in the second source operand. This operation has a 2 cycle latency.

3.2.2.80 `vec_half2 vec_v2shl (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Shift Left.

Logically shift the four 16-bit quantities in the first source operand to the left by the second source operand. The effective shift amount is the specified operand modulo 16. Logical left shift shifts zeros into the low ordered bits. This operation has a 1 cycle latency.

3.2.2.81 `vec_half2 vec_v2shli (vec_half2 SrcA, unsigned int ShAmt)`

Vector Two Byte Shift Left Immediate.

Logically shift the four 16-bit quantities in the first source operand to the left by an immediate. The effective shift amount is the specified immediate modulo 16. Left shifts shift zeros into the low ordered bits and is suitable to be used as unsigned multiplication by powers of two. This operation has a 1 cycle latency.

3.2.2.82 `vec_half2 vec_v2shlsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Shift Left Signed Clamped.

Logically shift the four 16-bit quantities in the first source operand to the left by the second source operand. The effective shift amount is the specified operand modulo 16. Logical left shift shifts zeros into the low ordered bits. If the left shift would arithmetically overflow a 16-bit quantity, the result is clamped to the minimum negative or maximum positive 16-bit value. This operation has a 1 cycle latency.

3.2.2.83 `vec_half2 vec_v2shrs (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Shift Right Signed.

Arithmetically shift the four 16-bit quantities in the first source operand to the right by the second source operand. The effective shift amount is the specified operand modulo 16. Arithmetic right shift shifts the high ordered bit into the high ordered bits. This operation has a 1 cycle latency.

3.2.2.84 `vec_half2 vec_v2shrsi (vec_half2 SrcA, unsigned int ShAmt)`

Vector Two Byte Shift Right Signed Immediate.

Arithmetically shift four 16-bit quantities in the first source operand to the right by an immediate. The effective shift amount is the specified immediate modulo 16. Arithmetic right shifts shift the source high ordered bit into the high ordered bits of the result. This operation has a 1 cycle latency.

3.2.2.85 `vec_half2 vec_v2shru (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Shift Right Unsigned.

Logically shift the four 16-bit quantities in the first source operand to the right by the second source operand. The effective shift amount is the specified operand modulo 16. Logical right shift shifts zeros into the high ordered bits. This operation has a 1 cycle latency.

3.2.2.86 `vec_half2 vec_v2shrui (vec_half2 SrcA, unsigned int ShAmt)`

Vector Two Byte Shift Right Unsigned Immediate.

Logically shift the four 16-bit quantities in the first source operand to the right by an immediate. The effective shift amount is the specified immediate modulo 16. Logical right shifts shift zeros into the high ordered bits. This operation is suitable to be used as unsigned integer division by powers of two. This operation has a 1 cycle latency.

3.2.2.87 `vec_half2 vec_v2sub (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Subtract.

Subtract the four 16-bit quantities in the second source operand from the four 16-bit quantities in the first source operand. This operation has a 1 cycle latency.

3.2.2.88 `vec_half2 vec_v2subsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Two Byte Subtract Signed Clamped.

Subtract the four 16-bit quantities in the second source operand from the four 16-bit quantities in the first source operand and clamp each result to the minimum negative value or maximum positive value. This operation has a 1 cycle latency.

3.2.2.89 `vec_half2 vec_v4add (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Add.

Add the two 32-bit quantities in the first source operand to the two 32-bit quantities in the second source operand. This operation has a 1 cycle latency.

3.2.2.90 `vec_half2 vec_v4addsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Add Signed Clamped.

Add the two 32-bit quantities in the first source operand to the two 32-bit quantities in the second source operand and clamp each result to the minimum negative or maximum positive 32-bit value. This operation has a 1 cycle latency.

3.2.2.91 `vec_half2 vec_v4int_h (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Interleave High.

Interleave the high-order 32-bit quantity of the first operand with the high-order 32-bit quantity of the second operand. The high-order 32-bits of the result will be the high-order 32-bits of the first operand. For example if the first operand contains the packed 32-bit quantities {A1,A0} and the second operand contains the packed 32-bit quantities {B1,B0} then the result will be {A1,B1}. This operation has a 1 cycle latency.

3.2.2.92 `vec_half2 vec_v4int_l (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Interleave Low.

Interleave the low-order 32-bit quantity of the first operand with the low-order 32-bit quantity of the second operand. The low-order 32-bits of the result will be the low-order 32-bits of the second operand. For example if the first operand contains the packed 32-bit quantities {A1,A0} and the second operand contains the packed 32-bit quantities {B1,B0} then the result will be {A0,B0}. This operation has a 1 cycle latency.

3.2.2.93 `vec_half2 vec_v4packsc (uint32_t SrcA, uint32_t SrcB)`

Vector Four Byte Pack Signed Clamped.

Clamp each of the packed 32-bit quantities in each of the two source registers to the maximum positive or minimum negative 16-bit value, and then pack the results into the destination register. The low-order 16-bit quantity of the destination will be the clamped low-order 32-bit quantity from the second operand. This operation has a 1 cycle latency.

3.2.2.94 `vec_half2 vec_v4shl (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Shift Left.

Logically shift the two 32-bit quantities in the first source operand to the left by the second source operand. The effective shift amount is the specified operand modulo 32. Logical left shift shifts zeros into the low ordered bits. This operation has a 1 cycle latency.

3.2.2.95 `vec_half2 vec_v4shlsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Shift Left Signed Clamped.

Logically shift the two 32-bit quantities in the first source operand to the left by the second source operand. The effective shift amount is the specified operand modulo 32. Logical left shift shifts zeros into the low ordered bits. If the left shift would arithmetically overflow a 32-bit quantity, the result is clamped to the minimum negative or maximum positive 32-bit value. This operation has a 1 cycle latency.

3.2.2.96 `vec_half2 vec_v4shrs (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Shift Right Signed.

Arithmetically shift the two 32-bit quantities in the first source operand to the right by the second source operand. The effective shift amount is the specified operand modulo 32. Arithmetic right shift shifts the high ordered bit into the high ordered bits. This operation has a 1 cycle latency.

3.2.2.97 `vec_half2 vec_v4shru (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Shift Right Unsigned.

Logically shift the two 32-bit quantities in the first source operand to the right by the second source operand. The effective shift amount is the specified operand modulo 32. Logical right shift shifts zeros into the high ordered bits. This operation has a 1 cycle latency.

3.2.2.98 `vec_half2 vec_v4sub (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Subtract.

Subtract the two 32-bit quantities in the second source operand from the two 32-bit quantities in the first source operand. This operation has a 1 cycle latency.

3.2.2.99 `vec_half2 vec_v4subsc (vec_half2 SrcA, vec_half2 SrcB)`

Vector Four Byte Subtract Signed Clamped.

Subtract the two 32-bit quantities in the second source operand from the two 32-bit quantities in the first source operand and clamp each result to the minimum negative value or maximum positive value. This operation has a 1 cycle latency.

3.3 Unaligned Data Access

3.3.1 Detailed Description

Routines for helping with accessing unaligned memory.

Data Structures

- struct `vec_UnalignedPointer`
An object corresponding to an unaligned pointer.
- struct `vec_UnalignedArray`
An object corresponding to an unaligned array of `uint64_t`.
- struct `_vec_UnalignedStruct`
An internal type used for the implementation of `vec_unaligned_load()`.

Defines

- `#define vec_init_unaligned_pointer(result, start_ptr)`
- `#define vec_init_unaligned_array(array, start_ptr)`
- `#define vec_unaligned_array_load_last(array)`
- `#define vec_unaligned_array_load_next(array)`
- `#define vec_unaligned_load(ptr)`
- `#define vec_unaligned_load_1(out0, uptr)`
- `#define vec_unaligned_load_2(out0, out1, uptr)`
- `#define vec_unaligned_load_4(out0, out1, out2, out3, uptr)`

3.3.2 Define Documentation

3.3.2.1 `#define vec_init_unaligned_pointer(result, start_ptr)`

Initializes `vec_UnalignedPointer` 'result' to point to 'start_ptr', a possibly unaligned pointer.

3.3.2.2 `#define vec_init_unaligned_array(array, start_ptr)`

Declares and initializes an unaligned array, so you can stride through it with `vec_unaligned_array_load_next`.

Note that, because of the way this is pipelined, this may end up reading one word past the end of your array. So make sure there is valid memory there.

Examples:

[Unaligned_Data_Access.](#)

3.3.2.3 #define vec_unaligned_array_load_last(array)

Returns the last unsigned integer from an unaligned array.

It does not advance the pointer to the next value or load any values. You do not need to use this to read the last array element, but it is a little cleaner to do so if you already know a specific load will be the last one.

3.3.2.4 #define vec_unaligned_array_load_next(array)

Returns the next unsigned integer from an unaligned array and advances the pointer to the next value.

Note that, because of the way this is pipelined, this may end up reading one word past the end of your array. So make sure there is valid memory there.

Examples:

[Unaligned_Data_Access.](#)

3.3.2.5 #define vec_unaligned_load(ptr)

Loads an unaligned long from an arbitrary address.

This is the most efficient way to load one unaligned value. If you are going to load several values with the same alignment, consider using [vec_init_unaligned_pointer\(\)](#) and [vec_unaligned_load_\[124\]](#), or for an array use [vec_init_unaligned_array\(\)](#) and [vec_unaligned_array_load_next\(\)](#).

Examples:

[Unaligned_Data_Access.](#)

3.3.2.6 #define vec_unaligned_load_1(out0, uptr)

Loads one `uint64_t` from the given [vec_UnalignedPointer](#) and stores it in out0.

3.3.2.7 #define vec_unaligned_load_2(out0, out1, uptr)

Loads two `uint64_t` values from the given [vec_UnalignedPointer](#) and stores them in out0 and out1.

3.3.2.8 #define vec_unaligned_load_4(out0, out1, out2, out3, uptr)

Loads four `uint64_t` values from the given [vec_UnalignedPointer](#) and stores them in out0, out1, out2 and out3.

Chapter 4

Data Structure Documentation

4.1 `_vec_UnalignedStruct` Struct Reference

An internal type used for the implementation of `vec_unaligned_load()`.

```
#include <vec_ops.h>
```

Data Fields

- `uint64_t __vec_unaligned_val`

4.1.1 Detailed Description

An internal type used for the implementation of `vec_unaligned_load()`.

4.1.2 Field Documentation

4.1.2.1 `uint64_t _vec_UnalignedStruct::__vec_unaligned_val`

Field used for dereferencing an unaligned long word of memory.

The documentation for this struct was generated from the following file:

- `vec_ops.h`

4.2 `vec_Byte8Union` Union Reference

A union structure for convenient access to members of a `vec_byte8`.

```
#include <vec_ops.h>
```

Data Fields

- `vec_byte8` `packed`
- `uint8_t` `array` [8]

4.2.1 Detailed Description

A union structure for convenient access to members of a `vec_byte8`.

4.2.2 Field Documentation

4.2.2.1 `vec_byte8` `vec_Byte8Union::packed`

Accessor for the packed data type.

4.2.2.2 `uint8_t` `vec_Byte8Union::array`[8]

Array version for individual access.

The documentation for this union was generated from the following file:

- `vec_ops.h`

4.3 vec_Half4Union Union Reference

A union structure for convenient access to members of a `vec_half4`.

```
#include <vec_ops.h>
```

Data Fields

- `vec_half4` `packed`
- `int16_t` `array` [4]

4.3.1 Detailed Description

A union structure for convenient access to members of a `vec_half4`.

4.3.2 Field Documentation

4.3.2.1 `vec_half4` `vec_Half4Union::packed`

Accessor for the packed data type.

4.3.2.2 `int16_t` `vec_Half4Union::array`[4]

Array version for individual access.

The documentation for this union was generated from the following file:

- `vec_ops.h`

4.4 `vec_Int2Union` Union Reference

A union structure for convenient access to members of a `vec_int2`.

```
#include <vec_ops.h>
```

Data Fields

- `vec_int2` `packed`
- `int32_t` `array` [2]

4.4.1 Detailed Description

A union structure for convenient access to members of a `vec_int2`.

4.4.2 Field Documentation

4.4.2.1 `vec_int2 vec_Int2Union::packed`

Accessor for the packed data type.

4.4.2.2 `int32_t vec_Int2Union::array[2]`

Array version for individual access.

The documentation for this union was generated from the following file:

- `vec_ops.h`

4.5 vec_UnalignedArray Struct Reference

An object corresponding to an unaligned array of uint64_t.

```
#include <vec_ops.h>
```

Data Fields

- [vec_UnalignedPointer ptr](#)
- uint64_t [val0](#)
- uint64_t [val1](#)

4.5.1 Detailed Description

An object corresponding to an unaligned array of uint64_t.

This supports the operation of efficiently fetching the next value in a streaming fashion.

Examples:

[Unaligned_Data_Access.](#)

4.5.2 Field Documentation

4.5.2.1 vec_UnalignedPointer vec_UnalignedArray::ptr

The raw pointer to memory.

4.5.2.2 uint64_t vec_UnalignedArray::val0

value from software-pipelined prefetching.

4.5.2.3 uint64_t vec_UnalignedArray::val1

value from software-pipelined prefetching.

The documentation for this struct was generated from the following file:

- `vec_ops.h`

4.6 `vec_UnalignedPointer` Struct Reference

An object corresponding to an unaligned pointer.

```
#include <vec_ops.h>
```

Data Fields

- `const uint64_t * ptr`
- `const void * align`

4.6.1 Detailed Description

An object corresponding to an unaligned pointer.

Because it contains both an aligned pointer and the original alignment information, it can be dereferenced more efficiently.

The 'ptr' field of this object is always an aligned pointer, equal to the truncated version of the possibly unaligned pointer passed to `vec_init_unaligned_pointer`. It can be modified as long as the new value is also equal to zero mod 8. For example, suppose you initialized this object with the unaligned pointer X and wanted to dereference the unaligned pointer X + 16. Since X + 16 has the same alignment as X, you can reuse this alignment object simply by:

```
my_struct.ptr += 2; // add 2 uint64_t == 16 bytes
```

The alignment information remains valid.

4.6.2 Field Documentation

4.6.2.1 `const uint64_t* vec_UnalignedPointer::ptr`

Aligned pointer to memory.

4.6.2.2 `const void* vec_UnalignedPointer::align`

Initial pointer, used for `dblalign`.

The documentation for this struct was generated from the following file:

- `vec_ops.h`

Chapter 5

Example Documentation

5.1 Unaligned_Data_Access

This example demonstrates several ways of using the unaligned data access macros. The first function demonstrates the use of [vec_unaligned_load\(\)](#) to load a word from an address of arbitrary alignment.

The next two functions compute the sum of an array of unaligned unsigned integers. The first function, `sum_unaligned_ints()` uses the simplistic approach of calling [vec_unaligned_load\(\)](#) for each address in the array. The second, `sum_unaligned_ints_fast()`, improves performance by using the streaming unaligned loads interface. It uses the [vec_init_unaligned_array\(\)](#) function to initialize a [vec_UnalignedArray](#) object and then uses the [vec_unaligned_array_load_next\(\)](#) function to load words from the array in sequence.

This example does not demonstrate the use of more specialized unaligned load functions like [vec_unaligned_load_4\(\)](#).

```
#include <stdio.h>
#include <stdint.h>
#include <vec_ops.h>

/* The vec_unaligned_load() function allows a single word to be loaded
 * from an address of arbitrary alignment.
 */
uint64_t
load_one(void* p)
{
    return vec_unaligned_load(p);
}

/* Sum an unaligned array of unsigned integer values by loading each
 * individually.
 */
uint64_t
sum_unaligned_ints(const void* ptr, int num_ints)
{
    uint64_t sum = 0;
```

CHAPTER 5. EXAMPLE DOCUMENTATION

```
    for (int i = 0; i < num_ints; i++)
        sum += vec_unaligned_load(ptr + (i * sizeof(uint64_t)));

    return sum;
}

/* Sum an unaligned array of unsigned integer values using the faster
 * streaming load API. This interface takes advantage of overlapping
 * data from consecutive unaligned loads; it will issue only one load
 * instruction per call to vec_unaligned_array_load_next(). However,
 * this interface will prefetch a few bytes past the end of the array,
 * so the user must be sure that the extra loads will not cause a
 * protection violation.
 */
uint64_t
sum_unaligned_ints_fast(const void* ptr, int num_ints)
{
    vec_UnalignedArray my_array;
    vec_init_unaligned_array(my_array, ptr);

    uint64_t sum = 0;
    for (int i = 0; i < num_ints; i++)
        sum += vec_unaligned_array_load_next(my_array);

    return sum;
}

int
main()
{
    /* This buffer is word-aligned. */
    char buf[1000];

    /* Initialize the buffer contents. */
    for (unsigned int i = 0; i < sizeof buf; i++)
        buf[i] = i;

    /* Load one unaligned word from inside buf. */
    printf("0x%llx\n", load_one(&buf[33]));

    /* Sum 10 words starting at buf + 17 (an unaligned address). */
    printf("%llu\n", sum_unaligned_ints(&buf[17], 10));

    /* Sum 10 words starting at buf + 17 (an unaligned address). */
    printf("%llu\n", sum_unaligned_ints_fast(&buf[17], 10));

    return 0;
}
```

5.2 Vector_Add

This example demonstrates use of the vector API, computing a vector sum of two arrays of 8-bit integers. In this simple example, the arrays must be 32-bit aligned.

```
#include<vec_ops.h>

void aligned_array_add(char* c, char* a, char* b, int n) {
    int* a_vec = (int*) a;
    int* b_vec = (int*) b;
    int* c_vec = (int*) c;

    int i;

    int n_by_4 = n >> 2;

    for(i = 0; i < n_by_4; i++) {
        c_vec[i] = vec_addb(a_vec[i], b_vec[i]);
    }
}
```

Index

`__vec_unaligned_val`
 `_vec_UnalignedStruct`, 31
`_vec_UnalignedStruct`, 31
 `__vec_unaligned_val`, 31

`align`

`vec_UnalignedPointer`, 36

`array`

`vec_Byte8Union`, 32
 `vec_Half4Union`, 33
 `vec_Int2Union`, 34

`instructions`

`vec_v1add`, 9
 `vec_v1addi`, 9
 `vec_v1adduc`, 9
 `vec_v1adiffu`, 9
 `vec_v1avgu`, 10
 `vec_v1cmpeq`, 10
 `vec_v1cmpeqi`, 10
 `vec_v1cmples`, 10
 `vec_v1cmpleu`, 10
 `vec_v1cmplts`, 10
 `vec_v1cmpltsi`, 11
 `vec_v1cmpltu`, 11
 `vec_v1cmpltui`, 11
 `vec_v1cmpne`, 11
 `vec_v1ddotpu`, 11
 `vec_v1ddotpua`, 12
 `vec_v1ddotpus`, 12
 `vec_v1ddotpusa`, 12
 `vec_v1dotp`, 12
 `vec_v1dotpa`, 12
 `vec_v1dotpu`, 13
 `vec_v1dotpua`, 13
 `vec_v1dotpus`, 13
 `vec_v1dotpusa`, 13

`vec_v1int_h`, 13
 `vec_v1int_l`, 14
 `vec_v1maxu`, 14
 `vec_v1maxui`, 14
 `vec_v1minu`, 14
 `vec_v1minui`, 14
 `vec_v1mnz`, 14
 `vec_v1multu`, 15
 `vec_v1mulu`, 15
 `vec_v1mulus`, 15
 `vec_v1mz`, 15
 `vec_v1sadau`, 15
 `vec_v1sadu`, 15
 `vec_v1shl`, 16
 `vec_v1shli`, 16
 `vec_v1shrs`, 16
 `vec_v1shrsi`, 16
 `vec_v1shru`, 16
 `vec_v1shrui`, 17
 `vec_v1sub`, 17
 `vec_v1subuc`, 17
 `vec_v2add`, 17
 `vec_v2addi`, 17
 `vec_v2addsc`, 17
 `vec_v2adiffs`, 18
 `vec_v2avgs`, 18
 `vec_v2cmpeq`, 18
 `vec_v2cmpeqi`, 18
 `vec_v2cmples`, 18
 `vec_v2cmpleu`, 18
 `vec_v2cmplts`, 19
 `vec_v2cmpltsi`, 19
 `vec_v2cmpltu`, 19
 `vec_v2cmpltui`, 19
 `vec_v2cmpne`, 19
 `vec_v2dotp`, 20

-
- vec_v2dotpa, [20](#)
 - vec_v2int_h, [20](#)
 - vec_v2int_l, [20](#)
 - vec_v2maxs, [20](#)
 - vec_v2maxsi, [21](#)
 - vec_v2mins, [21](#)
 - vec_v2minsi, [21](#)
 - vec_v2mnz, [21](#)
 - vec_v2mulfsc, [21](#)
 - vec_v2muls, [22](#)
 - vec_v2mults, [22](#)
 - vec_v2mz, [22](#)
 - vec_v2packh, [22](#)
 - vec_v2packl, [22](#)
 - vec_v2packuc, [23](#)
 - vec_v2sadas, [23](#)
 - vec_v2sadau, [23](#)
 - vec_v2sads, [23](#)
 - vec_v2sadu, [23](#)
 - vec_v2shl, [23](#)
 - vec_v2shli, [24](#)
 - vec_v2shlsc, [24](#)
 - vec_v2shrs, [24](#)
 - vec_v2shrsi, [24](#)
 - vec_v2shru, [24](#)
 - vec_v2shrui, [25](#)
 - vec_v2sub, [25](#)
 - vec_v2subsc, [25](#)
 - vec_v4add, [25](#)
 - vec_v4addsc, [25](#)
 - vec_v4int_h, [25](#)
 - vec_v4int_l, [26](#)
 - vec_v4packsc, [26](#)
 - vec_v4shl, [26](#)
 - vec_v4shlsc, [26](#)
 - vec_v4shrs, [26](#)
 - vec_v4shru, [27](#)
 - vec_v4sub, [27](#)
 - vec_v4subsc, [27](#)
 - packed
 - vec_Byte8Union, [32](#)
 - vec_Half4Union, [33](#)
 - vec_Int2Union, [34](#)
 - ptr
 - vec_UnalignedArray, [35](#)
 - vec_UnalignedPointer, [36](#)
 - types
 - vec_byte8, [6](#)
 - vec_half4, [6](#)
 - vec_int2, [6](#)
 - unaligned
 - vec_init_unaligned_array, [28](#)
 - vec_init_unaligned_pointer, [28](#)
 - vec_unaligned_array_load_last, [29](#)
 - vec_unaligned_array_load_next, [29](#)
 - vec_unaligned_load, [29](#)
 - vec_unaligned_load_1, [29](#)
 - vec_unaligned_load_2, [29](#)
 - vec_unaligned_load_4, [29](#)
 - Unaligned Data Access, [28](#)
 - val0
 - vec_UnalignedArray, [35](#)
 - val1
 - vec_UnalignedArray, [35](#)
 - vec_byte8
 - types, [6](#)
 - vec_Byte8Union, [32](#)
 - array, [32](#)
 - packed, [32](#)
 - vec_half4
 - types, [6](#)
 - vec_Half4Union, [33](#)
 - array, [33](#)
 - packed, [33](#)
 - vec_init_unaligned_array
 - unaligned, [28](#)
 - vec_init_unaligned_pointer
 - unaligned, [28](#)
 - vec_int2
 - types, [6](#)
 - vec_Int2Union, [34](#)
 - array, [34](#)
 - packed, [34](#)
 - vec_unaligned_array_load_last
 - unaligned, [29](#)

INDEX

`vec_unaligned_array_load_next`
 unaligned, [29](#)
`vec_unaligned_load`
 unaligned, [29](#)
`vec_unaligned_load_1`
 unaligned, [29](#)
`vec_unaligned_load_2`
 unaligned, [29](#)
`vec_unaligned_load_4`
 unaligned, [29](#)
`vec_UnalignedArray`, [35](#)
 ptr, [35](#)
 val0, [35](#)
 val1, [35](#)
`vec_UnalignedPointer`, [36](#)
 align, [36](#)
 ptr, [36](#)
`vec_v1add`
 instructions, [9](#)
`vec_v1addi`
 instructions, [9](#)
`vec_v1adduc`
 instructions, [9](#)
`vec_v1adiffu`
 instructions, [9](#)
`vec_v1avgu`
 instructions, [10](#)
`vec_v1cmpeq`
 instructions, [10](#)
`vec_v1cmpeqi`
 instructions, [10](#)
`vec_v1cmples`
 instructions, [10](#)
`vec_v1cmpleu`
 instructions, [10](#)
`vec_v1cmplts`
 instructions, [10](#)
`vec_v1cmpltsi`
 instructions, [11](#)
`vec_v1cmpltu`
 instructions, [11](#)
`vec_v1cmpltui`
 instructions, [11](#)
`vec_v1cmpne`
 instructions, [11](#)
`vec_v1ddotpu`
 instructions, [11](#)
`vec_v1ddotpua`
 instructions, [12](#)
`vec_v1ddotpus`
 instructions, [12](#)
`vec_v1ddotpusa`
 instructions, [12](#)
`vec_v1dotp`
 instructions, [12](#)
`vec_v1dotpa`
 instructions, [12](#)
`vec_v1dotpu`
 instructions, [13](#)
`vec_v1dotpua`
 instructions, [13](#)
`vec_v1dotpus`
 instructions, [13](#)
`vec_v1dotpusa`
 instructions, [13](#)
`vec_v1int_h`
 instructions, [13](#)
`vec_v1int_l`
 instructions, [14](#)
`vec_v1maxu`
 instructions, [14](#)
`vec_v1maxui`
 instructions, [14](#)
`vec_v1minu`
 instructions, [14](#)
`vec_v1minui`
 instructions, [14](#)
`vec_v1mnz`
 instructions, [14](#)
`vec_v1multu`
 instructions, [15](#)
`vec_v1mulu`
 instructions, [15](#)
`vec_v1mulus`
 instructions, [15](#)
`vec_v1mz`
 instructions, [15](#)
`vec_v1sadau`

-
- instructions, [15](#)
 - vec_v1sadu
 - instructions, [15](#)
 - vec_v1shl
 - instructions, [16](#)
 - vec_v1shli
 - instructions, [16](#)
 - vec_v1shrs
 - instructions, [16](#)
 - vec_v1shrsi
 - instructions, [16](#)
 - vec_v1shrui
 - instructions, [16](#)
 - vec_v1shru
 - instructions, [16](#)
 - vec_v1shrui
 - instructions, [17](#)
 - vec_v1sub
 - instructions, [17](#)
 - vec_v1subuc
 - instructions, [17](#)
 - vec_v2add
 - instructions, [17](#)
 - vec_v2addi
 - instructions, [17](#)
 - vec_v2addsc
 - instructions, [17](#)
 - vec_v2adiffs
 - instructions, [18](#)
 - vec_v2avgs
 - instructions, [18](#)
 - vec_v2cmpeq
 - instructions, [18](#)
 - vec_v2cmpeqi
 - instructions, [18](#)
 - vec_v2cmples
 - instructions, [18](#)
 - vec_v2cmpleu
 - instructions, [18](#)
 - vec_v2cmplts
 - instructions, [19](#)
 - vec_v2cmpltsi
 - instructions, [19](#)
 - vec_v2cmpltu
 - instructions, [19](#)
 - vec_v2cmpltui
 - instructions, [19](#)
 - vec_v2cmpne
 - instructions, [19](#)
 - vec_v2dotp
 - instructions, [20](#)
 - vec_v2dotpa
 - instructions, [20](#)
 - vec_v2int_h
 - instructions, [20](#)
 - vec_v2int_l
 - instructions, [20](#)
 - vec_v2maxs
 - instructions, [20](#)
 - vec_v2maxsi
 - instructions, [21](#)
 - vec_v2mins
 - instructions, [21](#)
 - vec_v2minsi
 - instructions, [21](#)
 - vec_v2mnz
 - instructions, [21](#)
 - vec_v2mulfsc
 - instructions, [21](#)
 - vec_v2muls
 - instructions, [22](#)
 - vec_v2mults
 - instructions, [22](#)
 - vec_v2mz
 - instructions, [22](#)
 - vec_v2packh
 - instructions, [22](#)
 - vec_v2packl
 - instructions, [22](#)
 - vec_v2packuc
 - instructions, [23](#)
 - vec_v2sadas
 - instructions, [23](#)
 - vec_v2sadau
 - instructions, [23](#)
 - vec_v2sads
 - instructions, [23](#)
 - vec_v2sadu
 - instructions, [23](#)
 - vec_v2shl
 - instructions, [19](#)

INDEX

- instructions, [23](#)
- vec_v2shli
 - instructions, [24](#)
- vec_v2shlsc
 - instructions, [24](#)
- vec_v2shrs
 - instructions, [24](#)
- vec_v2shrsi
 - instructions, [24](#)
- vec_v2shru
 - instructions, [24](#)
- vec_v2shrui
 - instructions, [25](#)
- vec_v2sub
 - instructions, [25](#)
- vec_v2subsc
 - instructions, [25](#)
- vec_v4add
 - instructions, [25](#)
- vec_v4addsc
 - instructions, [25](#)
- vec_v4int_h
 - instructions, [25](#)
- vec_v4int_l
 - instructions, [26](#)
- vec_v4packsc
 - instructions, [26](#)
- vec_v4shl
 - instructions, [26](#)
- vec_v4shlsc
 - instructions, [26](#)
- vec_v4shrs
 - instructions, [26](#)
- vec_v4shru
 - instructions, [27](#)
- vec_v4sub
 - instructions, [27](#)
- vec_v4subsc
 - instructions, [27](#)
- Vector Data Types, [5](#)
- Vector Instructions, [7](#)